Universität Duisburg-Essen
Fakultät für Informatik
Prof. Dr.-Ing. Lucas Davi
Noah Kappert
Firas Khamis

UNIVERSITÄT DUISBURG ESSEN
Open-Minded

Übungsblatt 1

Vorlesung „Secure Software Systems“
Wintersemester 2026/27

Praktische Übung
Intel Assembler und Reverse Engineering

Ziel dieser praktischen Übung ist das Erlernen von ausgewählten Grundlagen der Intel (32-Bit und 64-Bit) Instruction Set Architecture (ISA). Dazu werden Sie disassemblierte Programme analysieren und selbst Assembler Code verfassen.

Hello World Assembler Programm

In dieser Aufgabe werden Sie ein „Hello World“ Assembler Programm analysieren. Das `hello-x86` Programm in der Intel 32-Bit ISA (für Linux i386) geschrieben. Gehen Sie in den in der ZIP-Datei enthaltenen Ordner `1_hello`. Um das Programm `hello-x86` im Debugger *gdb* zu starten gehen Sie wie folgt vor:

- Laden Sie das Programm in `gdb`:
  `$ gdb ./hello-x86`
- Um das Programm zu disassemblieren können Sie folgendes Kommando in `gdb` verwenden:
  `disassemble _start`
  Alternativ dazu können Sie auch *Cutter* verwenden.
- Starten Sie das Programm in `gdb`:
  `start`
  und „steppen“ Sie Instruktion für Instruktion durch das Programm:
  `stepi` (oder kurz: `si`) (*step instruction*)
- Verwenden Sie `kill` und `start` um das Programm im Debugger zu terminieren und neu zu starten.

Machen Sie sich Notizen (z.B. in einer Assembler Datei (`.S`) oder mittels der Kommentarfunktion von Cutter) zu dieser Aufgabe.

Aufgabe 1.

i) Disassemblieren Sie die Routine `_start` in `hello-x86` und erklären Sie kurz jede Instruktion.

ii) Debuggen Sie das Programm `hello-x86` mit dem Debugger `gdb` und bestimmen Sie die tatsächlichen Werte und Adressen in den Registern jeweils bevor die `int 0x80` Instruktion ausgeführt wird. Erklären Sie die Bedeutung der verschiedenen Register.

iii) Wie funktionieren Linux System-Calls auf x86 32-Bit? Wie werden Parameter übergeben? Wie wird der Rückgabewert übergeben?

Library Funktionen in Assembler

In dieser Aufgabe werden Sie Funktionen in Assembler implementieren, die dann von einem C Programm genutzt werden. Dazu müssen die Assembler Funktionen entsprechend der Calling Convention von C auf Linux (cdecl) implementiert werden. Das Programm `main.c` ist vorgegeben, Sie dürfen keine Änderungen an dieser C-Datei vornehmen. Das Programm `main` liest bis zu 64 Zeichen vom Standard Input (stdin) und schreibt diese dann auf den Standard Output (stdout). Der Rückgabewert des Programm ist die Länge des eingegebenen C-Strings:

```
$ printf "Hello World!\n" | ./main
Hello World!
$ echo $?  # Exit Code des letzten Programms
13
```

Zum Bearbeiten dieser Aufgabe gehen Sie wie folgt vor:

- Entpacken Sie die ZIP-Datei und gehen Sie in den Ordner `2_mini_libc`.
- Kompilieren, assemblieren und linken Sie die Programme mit dem Befehl: `make`. Für jede Änderung der Quelldatei `lib.asm` müssen Sie erneut assemblieren und linken, also erneut den Befehl `make` verwenden.
- Öffnen Sie `lib.asm` und implementieren Sie die folgenden Funktionen in Assembler.

```c
/* write() writes up to count bytes from the buffer pointed
 * buf to the file referred to by the file descriptor fd.
 */
size_t write(int fd, const void *buf, size_t count);

/* read() attempts to read up to count bytes from file
 * descriptor fd into the buffer starting at buf.
 */
size_t read(int fd, void *buf, size_t count);

/* The strlen() function calculates the length of the
 * string pointed to by s, excluding the terminating null byte ('\0').
 */
size_t strlen(char* s);
```

- Testen Sie das Programm:
  i) Manuell indem Sie das Programm auf der Kommandozeile verwenden,
  ii) Mit dem mitgelieferten Tests: `make test`.
- Kommentieren Sie den Assembler Source Code so, dass Sie ihn gut Erklären können.

Aufgabe 2.

i) Finden Sie heraus, wie auf x86 Parameter übergeben werden (`cdecl` Calling Conventions) und wie die Funktion `read` auf die übergebenen Parameter zugreifen kann:

- Starten Sie den Debugger mit `gdb ./main`.
- Setzen Sie einen Breakpoint `break read`.
- Starten Sie das Programm mit `run`.
- Das Programm wird jetzt am Anfang von `read` angehalten. Sehen Sie sich den Inhalt der Register und des Stacks an (gdb: `regs` und `stack`).

ii) Implementieren Sie die Funktionen `read` und `write` in der Datei `lib.asm`. Diese sollen sich äquivalent zu den Standard-C-Funktionen verhalten. Implementieren Sie die Funktionen als einfache Wrapper um die entsprechenden Linux x86 System Calls. Im Gegensatz zu den Standard C Funktionen müssen Sie kein Error-Handling implementieren.

iii) Wie werden Schleifen auf Assemblerebene implementiert? Analysieren Sie die `memset` Funktion und beschreiben Sie, wie die Schleife implementiert wurde. Nutzen Sie dafür auch die Graphansicht von Cutter.

iv) C-Strings werden als `char*` dargestellt, also ein Pointer auf einen `char` (1 Byte). Erklären Sie, wie ein C-String im Speicher aussieht und wie die Länge eines C-Strings festgestellt werden kann. Implementieren Sie die `strlen` Funktion, welche die Länge eines C-Strings berechnet, in der Datei `lib.asm`. Testen Sie die Implementierung mit `make test`. Die Implementierung muss alle der mitgelieferten Tests bestehen.

v) **(Optional)** Wenn Sie `read`, `write` und `strlen` korrekt implementiert haben, dann befindet sich ein *buffer overread* Fehler (ähnlich zu dem bekannten HeartBleed-Bug) in dem `main` Programm. Finden Sie den Fehler und nutzen Sie GDB um den Fehler nachzuvollziehen. Welcher Input löst den Fehler aus? Was gibt das Programm dann aus? Wo befindet sich der Fehler? Was ist das zugrundeliegende Problem?

vi) **(Optional)** Bearbeiten Sie die Datei `main-patched.c` und bessern Sie den buffer overread Fehler aus. Dabei soll die originale Funktionalität weitestgehend gleich bleiben.

- Kompilieren Sie mit `make main-patched`
- Testen Sie die Funktionalität mit `make test-patched`
- Testen Sie, ob diese Version noch immer anfällig ist.

Links: Nützliche Links zum Thema Assembler Programmierung:

- `http://www.cs.virginia.edu/~evans/cs216/guides/x86.html`
- `http://unixwiz.net/techtips/x86-jumps.html`
- `http://www.felixcloutier.com/x86/`

Mehr Informationen zu den libc-Funktionen finden Sie auch in dem Man-pages:

- `man 2 read` oder `https://linux.die.net/man/2/read`
- `man 2 write` oder `https://linux.die.net/man/2/write`
- `man strlen` oder `https://linux.die.net/man/3/strlen`

Control Flow Analyse auf x86 64-Bit

In dieser Aufgabe ist Ihnen das Programm `cfa` gegeben. Dieses Programm benutzt die 64-Bit Variante der x86 ISA. Lösen sie diese Aufgabe mit Hilfe des vorgestellten Tools `Cutter`. Die Graph-Ansicht von *Cutter* kann beim Lösen dieser Aufgabe hilfreich sein.
Zum Bearbeiten dieser Aufgabe gehen Sie erst wie folgt vor:

- Öffnen Sie `cfa` in Cutter.
- Finden Sie die `explain_me` Funktion.

Aufgabe 3.

i) Erklären Sie, wie auf x86 64-Bit Parameter übergeben werden (*cdecl* Calling Conventions auf Linux). Erklären Sie auch, wie der Rückgabewert übergeben wird.

Finden Sie heraus wie viele Parameter an die Funktion `explain_me` übergeben werden und ob die Funktion einen Rückgabewert hat. Beschreiben Sie ihr Vorgehen.

ii) Erklären Sie kurz die Funktionalität von allen Basic Blocks in der Funktion `explain_me` im Programm `cfa`. Nutzen Sie zum Finden der Basic Blocks die Graph-Funktion von Cutter. Fügen Sie in Cutter Kommentare mit der Beschreibung der Basic Blocks ein. (Rechtsklick → *Add Comment* oder mit `;` als Tastaturkürzel)

iii) **(Optional)** Übersetzen Sie die Funktion `explain_me` in eine C Funktion. Erweitern Sie dafür die Vorlage `cfa_reversed.c`, welche bereits die fertige main Funktion enthält. Das resultierende Programm soll funktional äquivalent zu dem `cfa` Programm sein (Testen Sie mit `make test`). Können Sie erkennen unter welchem Namen die Funktion `explain_me` allgemein bekannt ist?

Anhang

A Linux System Calls für x86 32-bit

Tabelle 1 zeigt einen vereinfachten Ausschnitt aus der „System Call Table“ des Linux Kernels. Darin sollten sich alle wichtigen System Calls befinden, die Ihnen während der Übungen begegnen.

Tabelle 1: First 14 Linux Systems Calls on x86 32-bit

Number	Name	eax	ebx	ecx	edx	esi	edi
0	sys_restart_syscall	0x00	-	-	-	-	-
1	sys_exit	0x01	int error_code	-	-	-	-
2	sys_fork	0x02	struct pt_regs *	-	-	-	-
3	sys_read	0x03	unsigned int fd	char *buf	size_t count	-	-
4	sys_write	0x04	unsigned int fd	char *buf	size_t count	-	-
5	sys_open	0x05	char *filename	int flags	int mode	-	-
6	sys_close	0x06	unsigned int fd	-	-	-	-
7	sys_waitpid	0x07	pid_t pid	int *stat_addr	int options	-	-
8	sys_creat	0x08	char *pathname	int mode	-	-	-
9	sys_link	0x09	char *oldname	char *newname	-	-	-
10	sys_unlink	0x0a	char *pathname	-	-	-	-
11	sys_execve	0x0b	char* filename	char** argv	char** envp	struct pt_regs *	-
12	sys_chdir	0x0c	char *filename	-	-	-	-
13	sys_time	0x0d	time_t *tloc	-	-	-	-
14	sys_mknod	0x0e	char *filename	int mode	unsigned dev	-	-

B The ASCII Encoding

In C, strings are bytes that follow the ASCII encoding. Each numeric byte value corresponds to a character, some of which are printable, while others have special meaning, such as the null terminator or the newline character. Originally, ASCII was restricted to 7 bits, but was later extended to the full 8 bits/a single byte. In C, binary and printable data is often intermixed and this lack of distinction becomes important when analyzing or exploiting low-level vulnerabilities.

The full conversion table can be found in Figure 1.

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Abbildung 1: ASCII Table

```
# the following command can be typed into an interactive
# python shell (i.e., the "python" command or the advanced "ipython" shell)

# converting from numeric to a char
>>> chr(0x41)
'A'
# the reverse operation
>>> ord('A')
65
>>> hex(65)
'0x41'
# Indexing byte strings in python gives a numeric value
>>> b"A"[0]
65
```

Abbildung 2: Useful python functions for converting between numeric and ASCII character.